

Vorlesung Rechnerarchitekturen (AIN 2 – SoSe2022) Klausur (K90)

Datum: 07. Juli 2022, 16:45 Uhr

Bearbeitungszeit: 90 Minuten (Online)

Gesamtpunkte: 75

Mindestpunktzahl zum Bestehen der Klausur: 30

Name:

Matrikelnummer:

Semester:

Aufgabe	1	2	3	4	5	6	7	Gesamt
Punkte	10	5	8	12	11	12	17	75

Hinweise:

- Bei jeder angegebenen Lösung muss der Lösungsweg nachvollziehbar sein.
- Eingereichte Skripte gelten **nicht** als Lösungsweg!
- Erlaubte Hilfsmittel sind alle außer Quellen im Internet, die eine explizite Lösung oder Teillösung zu Klausuraufgaben bereitstellen, insbesondere natürlich von Dritten bereitgestellt Lösungen oder über das Internet verfügbare Programme, die Aufgaben oder Teilaufgaben lösen. Allgemeine Dienste wie z.B. Internet-Taschenrechner, Python-IDE, Binär-Dezimalrechner, etc. sind erlaubt.

Aufgabe 1 [Assembler Programmierung] (10 Punkte)

Implementieren Sie die folgende Funktion in MIPS Assembler. Halten Sie sich bei der Implementierung an die MIPS-Konventionen.

```
int f(int A[], int a, int i, int j) {  
    int m=a+4;  
    return A[m]+g(i,h(a,j));  
}
```

Die Funktionen `int g(int,int)` und `int h(int,int)` befinden sich an den Speicheradressen mit Label G und Label H. Sie können davon ausgehen, dass die Funktionen entsprechend der MIPS-Konventionen implementiert sind. (10P)

Aufgabe 2: Assembler Instruktionen (5 Punkte)

Die folgenden Tabellen enthalten eine Reihe von Instruktionen, die Sie nacheinander für die ebenfalls in den Tabellen gegebenen Register- und Speicherinhalte ausführen sollen. Tragen Sie die Veränderungen der gelisteten Register- und Speicherinhalte jeweils in den freien Feldern der Tabellen ein.

Hinweise:

- Punkte pro Instruktion wie in der ersten Spalte der Tabelle angegeben.

		Register (Inhalte als Signed Integer)						
		\$s0	\$s1	\$s2	\$t0	\$t1	\$t2	\$sp
P	Instruktionen	168	4	-1	1	17	-2	0x7FFF AF00
0,5	or \$t2, \$s1, \$t0						5	
1,0	lb \$t2, -5(\$t1)						-3	
0,5	addi \$t1, \$s0, 42					210		
1,0	lw \$s1, 4(\$s1)		272					
0,5	srl \$t1, \$s0, \$t0					84		
1,0	sw \$s2, 12(\$sp)							
0,5	sra \$t0, \$s0, 2				42			

Speicherausschnitt					
Adresse (hexadezimal)	Inhalt (unsigned Bytes)		Adresse (hexadezimal)	Inhalt (unsigned Bytes)	
...		Änderung	...	...	Änderung
0x0000 000F	0		0x7FFF AF13	0	
0x0000 000E	0		0x7FFF AF12	1	
0x0000 000D	0		0x7FFF AF11	0	
0x0000 000C	253		0x7FFF AF10	128	
0x0000 000B	0		0x7FFF AF0F	2	
0x0000 000A	0		0x7FFF AF0E	1	
0x0000 0009	1		0x7FFF AF0D	0	
0x0000 0008	16		0x7FFF AF0C	42	255
0x0000 0007	255		0x7FFF AF0B	7	0
0x0000 0006	255		0x7FFF AF0A	128	0
0x0000 0005	8		0x7FFF AF09	124	0
0x0000 0004	0		0x7FFF AF08	0	0

Aufgabe 3: Maschinensprache (8 Punkte)

Im Folgenden ist ein Stück Programm-Code sowohl in Assemblersprache als auch in Maschinensprache gegeben. Beide Programm-Codes weisen Lücken auf. Ergänzen Sie diese Lücken.

Hinweis: Punkte pro Instruktion in der rechten Spalte.

Verwenden Sie für diese Aufgabe die folgenden **OPCODES** und **FUNCTION** Codes:

Code (binär)	000001	000010	000011	000100	000101
Code ist OPCODE von Instruktion	addi	slti	ori	andi	nori
Code ist FUNCT von Instruktion	add	sub	slt	sllv	or
Code (binär)	000110	000111	001000	001001	001010
Code ist OPCODE von Instruktion	lw	sw	beq	bne	j
Code ist FUNCT von Instruktion	and	nor	sra	sll	srl

Verwenden Sie die folgende Registeradressierung:

Nummer	Adresse	Register	Nummer	Adresse	Register	Nummer	Adresse	Register
4	00100	\$t0	8	01000	\$s0	12	01100	\$a0
5	00101	\$t1	9	01001	\$s1	13	01101	\$a1
6	00110	\$t2	10	01010	\$s2	14	01110	\$a2
7	00111	\$t3	11	01011	\$s3	15	01111	\$a3

Füllen Sie die folgenden Lücken in Assembler- bzw. Maschinencode aus:

Speicher- adresse	Maschinenformat								Assembler	P
16	0001	1000	110_0	0101	1111	1111	1111	1111	L1: lw \$t1, -1(\$t2)	2
20	0000	0000	0000	0111	0111	0011	10 00	1000	L2: sra \$a2 \$t3 14	1
24	0000	0101	0000	1000	0000	0000	0000	1010	L3: addi \$s0, \$s0, 10	1
28	0010	00_01	0010	1101	1111	1111	1111	1100	L4: beq \$a1 \$s1 L2	2
32	0000	0000	111_0	1100	0110	1_000	0000	0100	L5: sllv \$a1 \$a0 \$t3	1
36	0010	10_00	0000	0000	0000	0000	0000	0110	L6: j L3	1

Aufgabe 4: Datenpfad und Steuerwerk (12 Punkte)

4.1 In Abbildung 4 sind Datenpfad und Steuerwerk einer Eintakt-Implementierung der MIPS CPU dargestellt. Der Inhalt der Register ist in Tabelle 4.1 gegeben. Verwenden sie für diese Aufgabe das MIPS Green Sheet. (6P)

Einige Daten- und Steuersignale sind mit Nummern bzw. Buchstaben in Rechtecken versehen. Tragen Sie in Tabelle 4.1 die Werte ein, die diese Signale bei der Ausführung der Instruktion

Maschinencode:

Binär: 0001 0011 0011 0011 0000 0111 1101 0000

Hex: 13 33 07 D0

R-Format (binär): 000100 11001 10011 00000 11111 010000

R-Format (dezimal): 4 25 19 0 31 16

I-Format (binär): 000100 11001 10011 0000011111010000

I-Format (dezimal): 4 25 19 2000

J-Format (binär): 000100 11001100110000011111010000

J-Format (dezimal): 4 53675984

im nächsten Takt annehmen. Die Instruktion steht im Speicher an Adresse 75000.

Hinweise:

- Sie können die Signale in Binär-, Dezimal- oder Hexadezimaldarstellung angeben.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Der Ausgang des Datenspeichers ist nur bei Ladebefehlen ungleich 0.
- In Tabelle 4.2 und Tabelle 4.3 sind die relevanten ALU Steuerbefehle gegeben.
- Der Code steht noch einmal auf der nächsten Seite

4.2 Eine MIPS CPU soll genau die folgenden Befehle unterstützen:

`sw, slti, slt, ori, andi, xori, and, lw, addi, bne, or, nori`

- Wie viele Bits muss dann das ALUOp Steuersignal (Control → ALU Control) mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)
- Wie viele Bits muss der ALU Steuerbefehl (ALU Control → ALU) dazu mindestens enthalten? Erklären Sie ihre Antwort. (1,5P)

4.3 Erweitern Sie Abbildung 4 so, dass die Instruktion `sll` unterstützt wird. (3P)

Name:

Matrikelnummer:

0	7	4	10	8	10	12	7	16	9	20	6	24	6	28	8
1	8	5	10	9	8	13	10	17	10	21	10	25	9	29	10
2	9	6	9	10	6	14	9	18	7	22	7	26	6	30	6
3	10	7	9	11	9	15	6	19	10	23	9	27	8	31	10

Tabelle 4.1: Registerinhalte in Hexadezimaldarstellung

ALU operation	Function	ALU operation	Function
0000	AND	0110	Subtract
0001	OR	0111	set-on-less-than
0010	Add	1100	NOR

Tabelle 4.2: Übersicht ALU Control

Instruction	ALUOp
lw, sw	00
Beq	01
R-Type	10

Tabelle 4.3: Übersicht ALUOp-Steuersignale

Daten- und Steuersignale					
1		4		A	
2		5		B	
3		6		memWrite	

Tabelle 4.4: Lösung

Maschinencode:

Binär: 0001 0011 0011 0011 0000 0111 1101 0000

Hex: 13 33 07 D0

R-Format (binär): 000100 11001 10011 00000 11111 010000

R-Format (dezimal): 4 25 19 0 31 16

I-Format (binär): 000100 11001 10011 0000011111010000

I-Format (dezimal): 4 25 19 2000

J-Format (binär): 000100 11001100110000011111010000

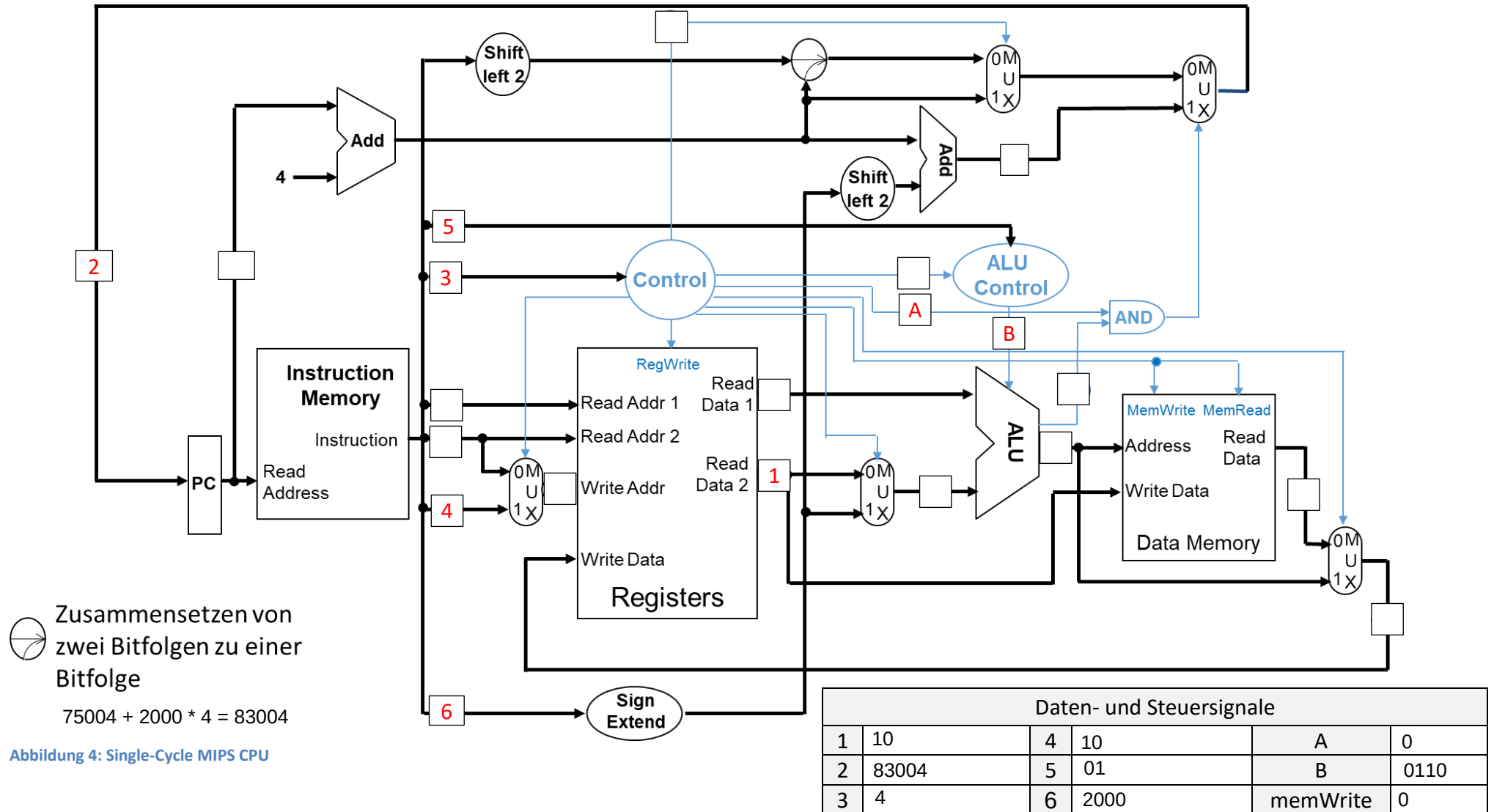
J-Format (dezimal): 4 53675984

Name:

Matrikelnummer:

R-Format (dezimal): 4 25 19 0 31 16

I-Format (dezimal): 4 25 19 2000



Aufgabe 5: Hazards (11 Punkte)

In dieser Aufgabe betrachten wir eine Multi-Cycle CPU mit Forwarding, auf der der Code in Tabelle 5 ausgeführt wird. Die CPU besitzt eine Einheit zur Branch-Prediction, so dass der neue Program-Counter bei Branch- und Jump-Instruktionen in der **IF-Stage** bestimmt und geschrieben wird. Außerdem verfügt die CPU über zusätzliche Hardware um Branch Bedingungen in der ID Stage auszuwerten.

- 5.1 Betrachten Sie die Data Hazards, die bei der Ausführung des Codes auftreten. Bestimmen Sie alle Data Hazards, die durch Forwarding beseitigt werden. Notieren Sie dazu in der mit „Forwarding“ bezeichneten Spalte das Forwarding in der Form $FW(\$x, R, S)$, wobei R das Pipeline-Register bezeichnet, aus dem der Operand genommen wird und S die Pipeline-Stage ist, in der das Register verwendet wird.

(7P)

Beispiel: $FW(\$s0, EX/MEM, EX)$

- 5.2 In Abbildung 5.1 sehen sie die Einheit um den nächsten Program-Counter für Branch und Jump-Instruktionen schon in der **IF-Stage** zu bestimmen. Beschreiben sie die einzelnen Komponenten und erklären sie ihre Aufgabe.

(4P)

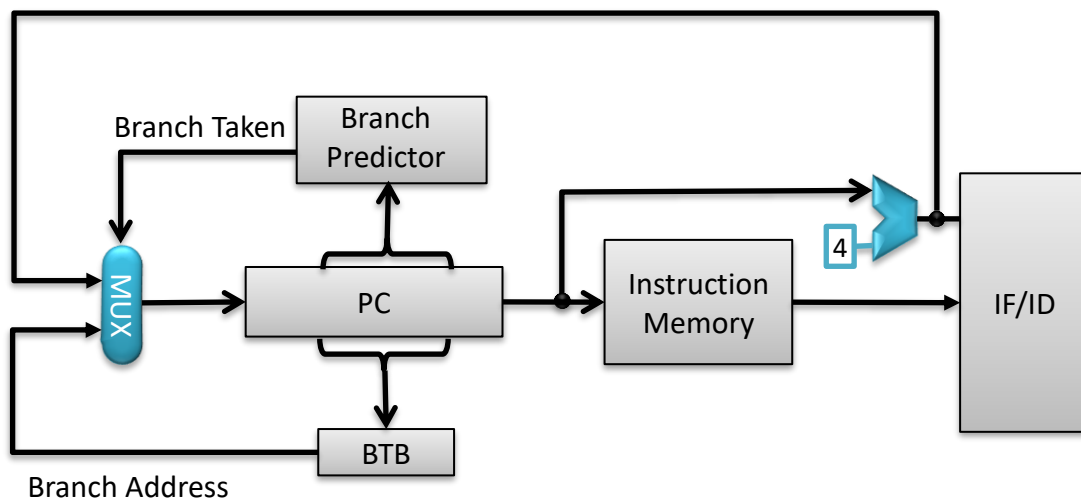


Abbildung 5.1: Einheit zur Bestimmung des nächsten Program-Counters für Branch- und Jump-Instruktionen

Name:

Matrikelnummer:

Tabelle 5: Code und Lösungs-Template zu Aufgabe 5

			Forwarding
1		add \$s2,\$t0,\$s1	
2		sw \$s2,8(\$s1)	FW(\$s2,EX/MEM,EX) FW(\$s2,MEM/WB,MEM)
3		bne \$s2,\$s1, L1	FW(\$s2,EX/MEM,ID)
4		j L2	
5	L1:	add \$s1,\$s2,\$t0	
6		lw \$s3,36(\$s1)	FW(\$s1,EX/MEM,EX)
7		lw \$t1,28(\$s1)	FW(\$s1,MEM/WB,EX)
8		add \$s2,\$s3,\$t1	FW(\$s3,MEM/WB,EX), FW(\$t1,EX/MEM,EX)
9		slti \$t0,\$s3,36	
10	L2:	sw \$t0,8(\$s2)	FW(\$t0,EX/MEM,EX)

Aufgabe 6: Pipeline (12 Punkte)

In Abbildung 6.2 ist die MIPS Pipeline mit Fokus auf die EX-Stage dargestellt. Der Code in Abbildung 6.1 wird auf der CPU ausgeführt. Die Register- und Speicherinhalte vor Ausführung von Zeile 1 sind in Tabelle 6.1 gegeben.

6.1 In der Darstellung der Pipeline finden Sie Zahlen in grau-markierten Kreisen. Tragen Sie in Tabelle 6.2 die entsprechenden Werte ein, wenn die **sw-Instruktion aus Zeile 5** in der EX-Stage ausgeführt wird. Dabei stehen die Zahlen für Pipeline-Register-Inhalte von ID/EX **vor** der Ausführung, Daten- und Steuersignale **während** der Ausführung und Pipeline-Register-Inhalte von EX/MEM **nach** der Ausführung der Instruktion in der EX-Stage. Gehen Sie davon aus, dass in dieser Aufgabe **keine Verzögerungen durch Control Hazards** auftreten. (9P)

Hinweise:

- In Abbildung 6.2 finden Sie auch Kopien der Abbildung 6.1 sowie der Tabellen 6.1 und 6.2, um eine einfachere Bearbeitung mit einem Bildschirm zu ermöglichen.
- Die Eingänge der Multiplexer sind von oben nach unten aufsteigend nummeriert. Geben Sie die Steuersignale als Dezimalwerte an.
- Alle **nicht relevanten** Steuerbefehle werden auf 0 gesetzt.
- Sie finden die **sw**-Instruktion in Binär, Dezimal- und Hexadezimalformat auf der nächsten Seite.
- Sie können alle Werte in Binär, Dezimal- oder Hexadezimalformat angeben. Achten Sie aber darauf, dass klar ist, welches Format Sie jeweils verwenden.

6.2 Bestimmen Sie den CPI einer Multi-Cycle CPU mit Branch-Prediction. Gehen Sie von den folgenden Parametern aus: (3P)

- Taktfrequenz: 3.5 GHz
- Anzahl Stages: 18
- CPI aller Instruktionen außer Branches: 1.6
- Branch-Prediction mit Branch-Target-Buffer in der zweiten Stage
- Anteil Branch-Instruktionen: 20%
- Anteil der richtigen Vorhersagen der Branch-Prediction: 91%
- Berechnung von Branch-Bedingung und Branch-Adresse erfolgt in der fünften Stage

$$0.8 * 1.6 + 0.2 * 0.91 * 3.0 + 0.2 * 0.09 * 5 = 1.916$$

Name:

Matrikelnummer:

Abbildung 6.1: Code-Ausschnitt

Zeile	Adresse	Instruktion
1	8000	add \$s0, \$s1, \$a0
2	8004	lw \$a2, 12(\$a0)
3	8008	lw \$a1, 4(\$s2)
4	8012	addi \$s0, \$t2, 16
5	8016	sw \$a2, 16(\$s0)

Assembler: sw \$a2, 16(\$s0)
Binär (Format): 001011 10000 00110 00000000000010000
Dezimal (Format): 11 16 6 16
Binär (je 4 Bit): 0010 1110 0000 0110 0000 0000 0001 0000
Hexadezimal: 2e 06 00 10

Tabelle 6.1: Register- und Speicherinhalt vor Ausführung von Zeile 1

Register	\$a0	\$s1	\$s2	\$t2
Inhalt (Word)	16	4	20	8
Speicher	16	20	24	28
Inhalt (Word)	12	12	12	12

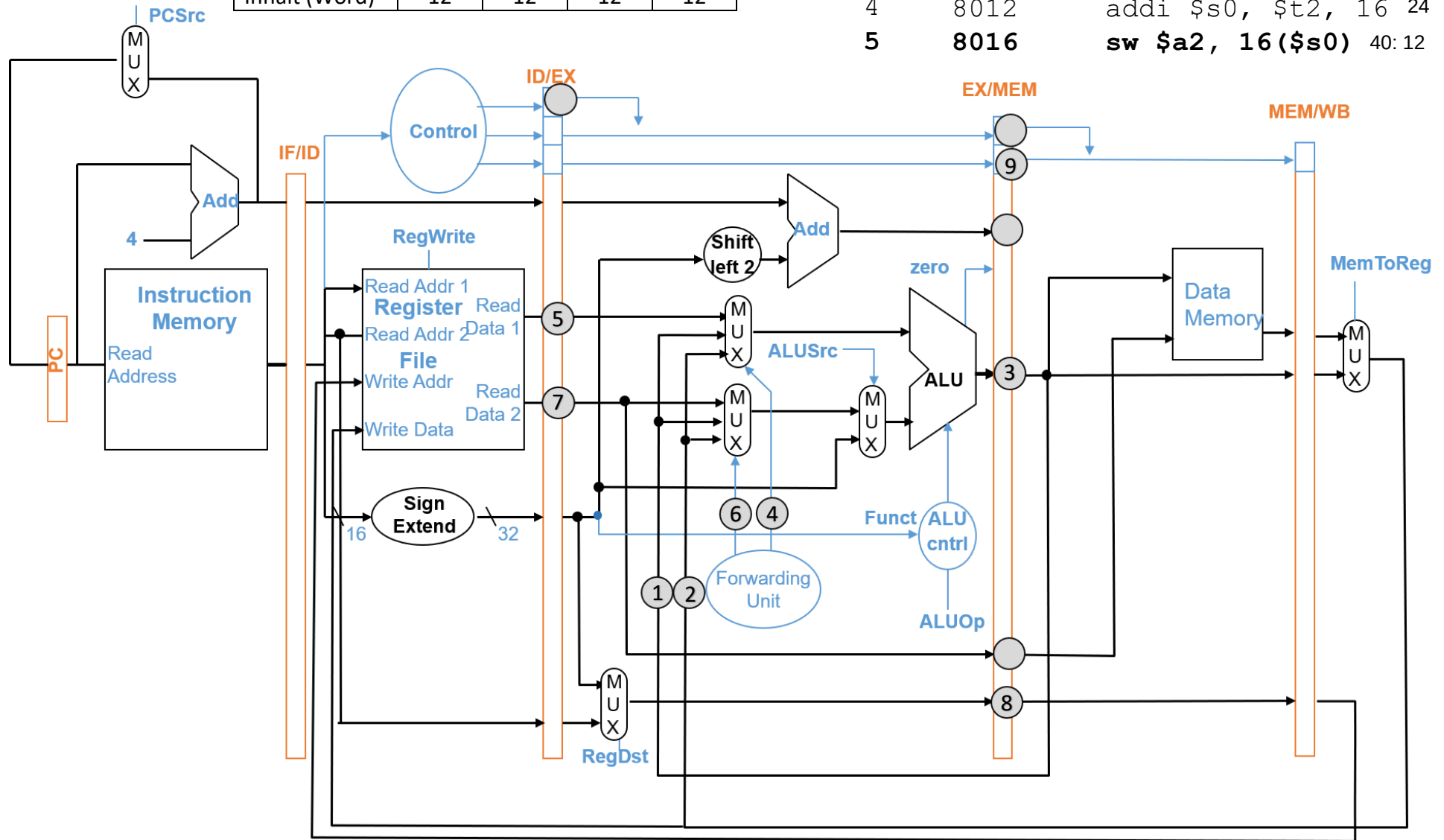
Tabelle 6.2: Registerinhalte, Daten- und Steuersignale bei Ausführung von "sw \$a2, 16(\$s0)" in EX

1		3		5		7	
2		4		6		8	
9							

Name:

Register	\$a0	\$s1	\$s2	\$t2
Inhalt (Word)	16	4	20	8
Speicher	16	20	24	28
Inhalt (Word)	12	12	12	12

Zeile	Adresse	Instruktion	
1	8000	add \$s0, \$s1, \$a0	20
2	8004	lw \$a2, 12(\$a0)	12
3	8008	lw \$a1, 4(\$s2)	12
4	8012	addi \$s0, \$t2, 16	24
5	8016	sw \$a2, 16(\$s0)	40: 12



1	FW EX/MEM: 24	3	EX/MEM ALU: 40	5	ID/EX Data1: 12	7	ID/EX Data2: 24
2	FW MEM/WB: 12	4	FW OP1: 0	6	FW OP2: 1	8	EX/MEM RegDst: 0
9	ALUSrc=1, ALUOp=ADD, RegDst=0						

Aufgabe 7: Cache (17 Punkte)

Ein Prozessor verwendet einen getrennten Daten- und Instruktionscache. Der Daten-Cache wird über eine Speicherhierarchie mit drei Cache-Ebenen realisiert. Auf allen drei Cache-Ebenen wird eine gemeinsame Blockgröße von 8 Bytes verwendet. Eine Übersicht der Cache-Hierarchie finden Sie in der folgenden Tabelle:

Level	Größe	Organisation	Schreib-Strategie
First	512 Bytes	4-way-set associative	Write-Back mit Allocate-on-miss
Second	1024 Bytes	2-way-set associative	
Third	65536 Bytes	8-way-set associative	

Ein Auszug des Inhalts der Caches ist in der Tabelle auf der nächsten Seite gegeben. Alle Caches verwenden eine LRU-Ersetzungsstrategie. Die Reihenfolge, in der die Blöcke ersetzt werden, wird über die Position des Blocks in der Tabelle vermerkt. Dabei wird der am weitesten rechts stehende Block zuerst ersetzt.

Die folgende Instruktion wird ausgeführt: `lw $s1, 0($sp)`

Vor der Ausführung steht im `$sp`-Register der folgende Wert:

Dezimal: 3428

Binär: 0000 0000 0000 0000 0000 1101 0110 0100

Hexadezimal: 00 00 0d 64

7.1 Bestimmen Sie, welcher Wert in das Register `$s1` geschrieben wird. Geben Sie alle Zwischenschritte an. (6P)

7.2 Tragen Sie in der leeren Tabelle die Änderungen der Cache-Inhalte ein, die durch die Ausführung der „lw“-Instruktion verursacht werden.

Hinweis: Wenn sich lediglich die Position eines Blocks im Set verändert, genügt es lediglich die Tags an der richtigen Position einzutragen. Ein Kopieren der Inhalte ist nicht notwendig. (7P)

7.3 Bestimmen Sie die Anzahl der Takte, die die Ausführung der „lw“-Instruktion benötigt, wenn ein Zugriff auf den First-Level-Cache 3 Takte, ein Zugriff auf den Second-Level-Cache 25 Takte, ein Zugriff auf den Third-Level-Cache 100 Takte und ein Zugriff auf den Hauptspeicher 1000 Takte benötigt. Der Zugriff für das Lesen und Schreiben eines Blocks dauert gleich lang. (4P)

$$3+25+100+25 = 153$$

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
10	484	Y	N	07 D1	1016	Y	N	BB AB	122	Y	N	A7 D2	229	Y	N	F4 E6
11	492	Y	N	CD A5	349	Y	N	4B 26	610	Y	N	11 71	545	N	N	04 83
12	146	Y	N	30 5B	572	Y	N	64 4A	1012	Y	N	1A 12	155	Y	Y	F3 5E
13	246	Y	N	BB BC	191	Y	N	BE 2E	372	Y	N	6F E6	90	Y	N	66 39
14	155	Y	N	CC F3	997	Y	N	D6 35	633	Y	N	CD 36	73	Y	N	51 17
15	609	Y	N	02 BB	489	Y	N	AB 9F	975	Y	N	FE CB	580	Y	N	05 6C

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
41	140	Y	N	8B 06	138	N	N	45 3C
42	30	Y	N	A7 D2	85	Y	N	AC B7
43	152	Y	N	11 71	30	N	N	07 48
44	36	Y	N	30 5B	38	Y	Y	F3 5E
45	61	Y	N	BB BC	22	N	N	66 39

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
56	144	Y	N	BB 78	243	N	N	D0 11
57	34	Y	N	51 CB	155	Y	N	BF 92
58	93	Y	N	32 79	87	N	N	B0 6A
59	129	Y	N	A9 65	116	N	N	75 FD
60	50	Y	N	28 32	137	N	N	47 ED

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
317	15	Y	N	FA 33	1	Y	Y	D9 B9	8	Y	N	B9 6D	9	Y	N	F8 26	3	Y	Y	86 3C	7	Y	N	60 37	5	Y	N	9A B6	2	Y	N	C4 07
318	1	Y	Y	B0 E4	7	Y	Y	AE 44	8	Y	Y	B4 96	3	Y	N	C5 08	2	Y	Y	77 CE	9	Y	N	93 E5	15	Y	N	6C CF	9	Y	Y	10 44
319	8	Y	Y	9E C5	9	Y	Y	F0 85	2	Y	N	CD 98	15	Y	N	E1 92	5	Y	Y	F6 73	3	Y	Y	5F 77	1	Y	N	5F AE	13	Y	N	3D D7
320	15	Y	N	B7 40	5	Y	Y	90 CD	8	Y	Y	E5 31	9	Y	Y	21 77	2	Y	N	88 8E	1	Y	Y	AF AC	1	Y	Y	6C 31	96	Y	Y	AE 28
321	15	Y	N	63 61	5	Y	N	0C 66	7	Y	N	7C 19	1	Y	N	14 D6	3	Y	N	09 4F	9	Y	Y	92 7C	2	Y	N	6B D3	8	Y	Y	33 BB

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
425	15	Y	N	49 91	7	Y	Y	8F 12	2	Y	Y	27 D5	3	Y	N	38 1B	8	Y	N	DF B2	5	Y	N	30 0D	1	Y	N	1D 66	9	Y	Y	90 02
426	5	Y	N	AB 57	2	Y	Y	95 26	9	Y	Y	31 28	1	Y	Y	DD 60	15	Y	Y	0B A0	8	Y	N	AD 4B	88	Y	Y	23 A7	60	Y	Y	D7 2E
427	2	Y	Y	38 1E	7	Y	Y	18 2D	5	Y	Y	7C AC	15	Y	N	A9 06	3	Y	Y	35 CA	9	Y	Y	5A 1A	8	Y	N	0E 86	93	Y	Y	C9 F0
428	2	Y	Y	F3 5E	8	Y	Y	94 57	3	Y	N	BE 4F	0	Y	N	6D 65	9	Y	N	40 F9	1	Y	N	9D 01	15	Y	N	97 09	7	Y	N	5A 8D
429	1	Y	Y	66 39	9	Y	Y	77 78	7	Y	Y	65 3D	5	Y	Y	05 0A	2	Y	N	38 6B	15	Y	Y	4E 4A	8	Y	Y	40 01	3	Y	N	2B 6A

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block				
443	15	Y	N	9F EF	8	Y	N	C4 95	1	Y	Y	62 A8	9	Y	Y	86 8E	3	Y	N	2F F1	7	Y	Y	21 34	2	Y	N	7F 66	5	N	Y	88 6C
444	8	Y	Y	05 1F	2	Y	N	51 9F	1	Y	N	93 20	3	Y	Y	C4 C1	5	Y	N	66 5D	9	Y	N	A5 48	7	Y	N	4F 08	15	Y	N	3B DA
445	9	Y	Y	72 F0	5	Y	N	05 9F	8	Y	Y	E9 99	3	Y	N	A2 ED	1	Y	N	76 57	7	N	N	E0 2C	17	Y	Y	07 EE	29	Y	N	B8 4E

Auszug First-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
10																
11																
12	26	1	0	6D 65	146				572				1012			
13																
14																
15																

Auszug Second-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block
41								
42								
43								
44	6			6D 65	36			
45								

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block
56								
57								
58								
59								
60								

Auszug Third-Level-Cache

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
317																												
318																												
319																												
320																												
321																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
425																												
426																												
427																												
428	0	1	0	6D 65	2				8				3															
429																												

Auszug ...

Index	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block	Tag	V	D	Block
443																												
444																												
445																												